

Aufgaben

1. Klassendefinition, Konstruktoren und Destruktoren

- Schreiben Sie eine Klasse zur Speicherung von Punkten mit x/y -Koordinaten und dem Namen des Punktes. Ergänzen Sie in der Klasse eine Methode zum Zeichnen des Punktes samt Namen mit der Turtle.
- Übernehmen Sie folgende Funktionen in Ihre Klassendefinition:

```
def __init__(self, name):  
    self.name = name  
    print(f"Objekt {self.name} erzeugt.")  
  
def __del__(self):  
    print(f"Objekt {self.name} gelöscht.")
```

Erzeugen Sie mehrere Objekte vom Typ `Punkt` und zeichnen Sie diese, indem Sie die programmierte Methode zum Zeichnen aufrufen.

Wann wird der Konstruktor aufgerufen und wann wird der Destruktor aufgerufen?

2. Erstellen und Nutzen einer eigenen Klasse

Ziel dieser Aufgabe ist die Erstellung einer eigenen Klasse zur Patientenverwaltung. Mit der Klasse `Patient` sollen beliebig viele Objekte vom Typ `Patient` erstellt werden können.

Erstellen Sie eine Klasse `Patient` mit folgenden (Objekt-)Variablen:

```
name  
alter  
gewicht  
groesse
```

Beim Erzeugen eines Objektes sollten die Variablen über den Konstruktor mit Werten gesetzt werden können, also z.B. soll eine Objekterzeugung wie folgt aussehen:

```
horst = Patient("Horst", 45, 88, 1.99)
```

womit im Objekt namens `horst` vom Typ `Patient` der Name "Horst", das Alter 45, das Gewicht 88 und die Größe 1.99 gespeichert werden.

3. Erweitern einer Klasse um Methoden

- Ergänzen Sie die Klasse `Patient` um eine Methode `ausgabe()`, welche alle Daten eines Objektes ausgibt.

Die Ausgabe kann z.B. wie folgt aussehen:

```
"Der Patient Horst ist 45 Jahre alt, wiegt 88 kg und ist 1.99m groß."
```

Testen Sie Ihre Methode, indem Sie einige Objekte (mindestens drei) dieser Klasse erzeugen und deren Daten ausgeben.

- Ergänzen Sie eine weitere Methode `berechneBMI()` zur Berechnung des Body Mass Index (BMI). Der BMI soll in einer Variable gespeichert werden und in der Methode `ausgabe()` zusammen mit den anderen Daten ausgegeben werden können.

Testen Sie Ihre neue Methode mit mehreren Objekten.

- Ergänzen Sie im Konstruktor den Aufruf der Methode `berechneBMI()`, damit der BMI automatisch beim Anlegen eines neuen Objektes berechnet wird.

Testen Sie Ihre neue Methode mit mehreren Objekten.

4. Optional: Anwenden des Geheimnis-Prinzips auf die Klasse `Patient`

- Erzeugen Sie zunächst ein Objekt der Klasse `Patient` und geben Sie die Werte auf dem Bildschirm aus.
- Ändern Sie nun über eine direkte Zuweisung den Wert der Variablen `gewicht` und geben Sie die Werte auf dem Bildschirm aus, also z.B.:

```
einPatient.gewicht = 77  
einPatient.ausgeben()
```

Was fällt Ihnen auf, wenn Sie den BMI vor und nach der Änderung vergleichen?

- Dies ist ein typischer Fall, wo es sinnvoll ist, Änderung von Werten nur über Methoden durchzuführen. Methoden bieten zum einen den Vorteil, übergebene Werte auf Gültigkeit zu prüfen (z.B. ob das Gewicht eine Zahl größer 0 ist) und damit verbundene Werte zu aktualisieren (wie z.B. den BMI, welcher bei einer Gewichtsänderung neu berechnet werden muss).

Ergänzen Sie in der Klasse `Patient` Methoden zum Ändern der Variablen `gewicht`, `groesse` und `alter`. Ergänzen Sie in den Methoden ggf. die Neuberechnung des BMI, sofern notwendig.

Testen Sie Ihre Methoden anhand von Objekten. Geben Sie hierzu vor und nach einer Änderung die Werte aus.

- Setzen Sie die Zugriffsmodifikatoren für die Variablen des Objektes auf `private`, damit man die Variablen nicht direkt ändern kann. Was passiert, wenn Sie nun einen Wert wie in Aufgabenteil b) ändern wollen?